GUIA TÉCNICO | 15

Data augmentation e avaliação robusta em visão

Capítulo 15/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Escolher transformações plausíveis e impedir vazamento entre imagens relacionadas.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Transformações de treino precisam preservar o rótulo. Flips podem ser inadequados em números e medicina; crops podem eliminar o objeto. Em dados com múltiplas imagens por pessoa, dividir por indivíduo antes de augmentar.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: X_aug=T(X_treino)

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Augmentation só pode agir sobre o conjunto de treino, após separação por grupo/indivíduo quando houver dependência.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
im=np.arange(16).reshape(4,4)
flip=np.fliplr(im);noise=np.random.default_rng(7).normal(0,.01,size=im.shape)
print('Original:',im[0].tolist(),'flip horizontal:',flip[0].tolist(),'ruído ↪
    média:',round(noise.mean(),4))
assert np.array_equal(flip[:,::-1],im)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Separação por grupo antes da transformação de treino

Augmentation só pode agir sobre o conjunto de treino, após separação por grupo/indivíduo quando houver dependência.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
from sklearn.model_selection import GroupShuffleSplit
rng=np.random.default_rng(15);img=rng.normal(size=(24,8,8));groups=np.repeat(np.arange(8),  ↪
    3)
split=GroupShuffleSplit(n_splits=1,test_size=.25,random_state=15)
tr,te=next(split.split(img,groups=groups))
assert not (set(groups[tr]) & set(groups[te]))
aug=img[tr]+rng.normal(0,.01,size=img[tr].shape)
print('Grupos treino:',len(set(groups[tr])),'teste:',len(set(groups[te])),'imagens  ↪
    aumentadas:',aug.shape[0])
```

Aumentar somente imagens do treino depois da divisão; testar se as transformações preservam o significado da classe.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que augmentação deve ocorrer após separar treino e teste?

Resposta: Para evitar versões transformadas da mesma imagem aparecendo em conjuntos diferentes.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências,

confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide